Table of Contents

1. Component Architecture Overview
2. Component Directory Structure
3. Core Components
   - 1. Authentication Components
     - MFAEnrollment
     - MFAVerification
   - 2. Canvas System
     - Canvas
     - Box Annotations
     - ToolbarColorSelector
     - Confirmation
   - 3. Sidebar System
     - Sidebar Container
     - Sidebar
     - Case Sidebar
     - Notes Sidebar
     - Cases Modal
     - Files Modal
     - Notes Modal
     - Case Export
     - Case Import
   - 4. Action Components
     - Case Management
     - Case Export
     - Case Import
     - Confirmation Export
     - Image Management
     - PDF Generation
     - Notes Management
     - Sign Out
   - 5. UI Components
     - Button System
     - Color System
     - Icon System
     - Public Signing Key Modal
     - Toast System
     - Toolbar
     - Theme Provider
   - 6. User Management Components
     - User Profile Management
     - Delete Account
     - Inactivity Warning
   - 7. Audit Trail Components
     - User Audit Viewer
     - Audit Export Service
4. Component State Management
   - Local State Patterns
   - Context Usage
     - AuthContext
   - Custom Hooks
     - useInactivityTimeout
     - Business Logic Hooks Pattern
     - useImportState
     - useFilePreview
     - useImportExecution
     - Custom Hooks Best Practices
5. Component Communication Patterns
   - Props Down, Events Up
   - Event Handling
   - Modal and Dialog Patterns

6. Styling Approach
    - CSS Modules
    - Style Conventions
7. Performance Considerations
    - Component Lifecycle
8. Accessibility Features
    - Built-in Accessibility
9. Development Guidelines
    - Component Creation Checklist
    - Best Practices

Component Architecture Overview

Striae's frontend is built using React components organized in a modular structure. This guide covers the major components, their purposes, and how they interact within the application.

Component Directory Structure

```
app/components/
├── actions/                    # Data handling components
├── audit/                      # Audit trail UI components
├── auth/                       # Authentication components
├── button/                     # Reusable button components
├── canvas/                     # Main canvas for image annotation
├── colors/                     # Color picker components
├── form/                       # Shared form/input primitives
├── icon/                       # Icon system
├── public-signing-key-modal/   # Public signing key display/copy modal
├── sidebar/                    # Sidebar navigation and controls
├── theme-provider/             # Theme management
├── toast/                      # Toast notification system
├── toolbar/                    # Main toolbar components
└── user/                       # User management components
```

Core Components

1. Authentication Components

MFAEnrollment (`app/components/auth/mfa-enrollment.tsx`)

Purpose: Multi-factor authentication setup

Features:

- Phone number verification
- SMS code validation
- Firebase MFA integration
- reCAPTCHA verification

Type Definition: Uses Firebase authentication types for MFA setup

Key Props:

- `user: User` - Firebase user object
- `onSuccess: () => void` - Success callback
- `onError: (message: string) => void` - Error callback
- `mandatory: boolean` - Whether MFA is required

MFAVerification (`app/components/auth/mfa-verification.tsx`)

Purpose: MFA challenge during login

Features:

- Multi-factor resolver handling
- SMS code input and validation
- Error handling and retry logic

Type Definition: Uses Firebase `MultiFactorResolver` interface for MFA challenges

2. Canvas System

Canvas (`app/components/canvas/canvas.tsx`)

Purpose: Main image display and annotation interface

Features:

- High-resolution image rendering
- Annotation overlay display
- Loading states and error handling
- Flash effects for user feedback (subclass characteristics)

Type Definition: Uses `AnnotationData` interface from `app/types/annotations.ts`

Key Props:

```
interface CanvasProps {
  imageUrl?: string;
  filename?: string;
  company?: string;
  firstName?: string;
  error?: string;
  activeAnnotations?: Set<string>;
  annotationData?: AnnotationData | null;
}
```

State Management:

- Image loading states
- Error handling for network issues
- Flash effects for user feedback (subclass characteristics)

Key Methods:

- Image load error detection
- Annotation overlay rendering
- User interaction handling

Box Annotations (`app/components/canvas/box-annotations/box-annotations.tsx`)

Purpose: Interactive box annotation drawing and management system

Features:

- Mouse-based box drawing with real-time visual feedback
- Percentage-based coordinate system for device independence
- Double-click and right-click removal functionality
- Hover effects with deletion indicators
- Transparent styling with colored borders
- Automatic saving integration with existing annotation system

Type Definition: Uses `BoxAnnotation` interface from `app/types/annotations.ts`

Key Props:

```
interface BoxAnnotationsProps {
  imageRef: React.RefObject<HTMLImageElement>;
  activeAnnotations?: Set<string>;
  annotationData?: AnnotationData | null;
  onSave: (data: AnnotationData) => void;
  selectedColor: string;
  isBoxAnnotationMode: boolean;
}
```

State Management:

- Drawing state tracking (isDrawing, startPosition, currentBox)
- Box annotation array management
- Real-time coordinate calculation and display
- Integration with toolbar visibility controls

Key Methods:

- `handleMouseDown`: Initiates box drawing on mouse press
- `handleMouseMove`: Updates current box dimensions during drawing
- `handleMouseUp`: Finalizes box creation and triggers save
- `handleDoubleClick` / `handleRightClick`: Box removal functionality
- `calculatePercentageCoordinates`: Converts pixel coordinates to percentages

ToolbarColorSelector (`app/components/toolbar/toolbar-color-selector.tsx`)

Purpose: Dynamic color selection interface for box annotations

Features:

- Preset color grid with common annotation colors
- Custom color wheel for precise color selection
- Confirm/cancel workflow with visual preview
- Automatic appearance when box annotation tool is active
- Reset functionality to restore previous color selection

Type Definition: Uses component-specific `ToolbarColorSelectorProps` interface

Key Props:

```
interface ToolbarColorSelectorProps {
  isVisible: boolean;
  selectedColor: string;
  onColorSelect: (color: string) => void;
  onConfirm: () => void;
  onCancel: () => void;
}
```

State Management:

- Temporary color selection state
- Preset color array management
- Visual preview of selected color
- Confirmation state tracking

Key Methods:

- `handleColorSelect`: Updates temporary color selection
- `handleConfirm`: Applies selected color and closes selector
- `handleCancel`: Reverts to previous color and closes selector
- `resetToDefault`: Resets color selection to default value

Confirmation (`app/components/canvas/confirmation/confirmation.tsx`)

Purpose: Authenticated confirmation modal for comparison image verification workflow

Features:

- Digital authentication with examiner credentials (badge ID and full name)
- Unique confirmation ID generation for audit trails
- Timestamp capture with human-readable and ISO formats
- View-only mode for existing confirmations
- Form validation requiring badge ID input
- Keyboard event handling (Escape key to close)
- Dual-mode operation: new confirmation creation or existing confirmation viewing
- Automatic population of user metadata (name, email, company)
- Integration with confirmation data structure for PDF export

Type Definition: Uses `ConfirmationData` interface from `app/types/annotations.ts`

Key Props:

```
interface ConfirmationModalProps {
  isOpen: boolean;
  onClose: () => void;
  onConfirm?: (confirmationData: ConfirmationData) => void;
  company?: string;
  existingConfirmation?: ConfirmationData | null;
}
```

Confirmation Data Structure:

```
interface ConfirmationData {
  fullName: string;           // Confirming examiner's full name
  badgeId: string;            // Badge/ID number of confirming examiner
  timestamp: string;          // Human-readable confirmation timestamp
  confirmationId: string;     // Unique ID generated at confirmation time
  confirmedBy: string;        // User UID of the confirming examiner
  confirmedByEmail: string;   // Email of the confirming examiner
  confirmedByCompany: string; // Company/Lab of the confirming examiner
  confirmedAt: string;        // ISO timestamp of confirmation
}
```

State Management:

- Badge ID input tracking with validation
- Error state for invalid or missing inputs
- Confirmation processing state (loading/submitting)
- Automatic form reset on modal open
- Detection and display of existing confirmation data

Key Methods:

- `handleConfirm`: Validates badge ID, generates confirmation data with unique ID, triggers callback
- `handleOverlayClick`: Closes modal when clicking outside content area
- `formatTimestamp`: Generates human-readable timestamp with full date and time
- `generateConfirmationId`: Creates unique confirmation identifier (via utility function)

UI Modes:

- **New Confirmation Mode**:
  - Editable badge ID field with autofocus
  - Active confirm button when badge ID is provided
  - Displays current user's credentials
  - Generates new confirmation ID and timestamp
- **View Mode** (when `existingConfirmation` provided):
  - All fields read-only
  - Green banner indicating confirmed status
  - Displays original confirming examiner's credentials
  - Close button instead of confirm/cancel actions
  - Shows preserved confirmation ID and timestamp

Security Features:

- Requires user authentication (Firebase user context)

- Captures user UID for confirmation attribution
- Generates unique IDs for confirmation tracking
- Immutable confirmation data once created
- ISO timestamp for precise temporal documentation

Integration Points:

- Connected to Notes Sidebar for "Include confirmation" checkbox workflow
- Confirmation data stored within `AnnotationData` structure
- Exported in case data for reviewing examiner import
- Appears in PDF reports with full confirmation details
- Part of forensic workflow for independent verification

Workflow Context:

1. Original examiner marks image with "Include confirmation" checkbox
2. Case exported and transferred to reviewing examiner
3. Reviewing examiner imports case in read-only mode
4. Upon independent verification, clicks "Confirm" button
5. This modal captures confirming examiner's credentials
6. Confirmation data exported back to original examiner
7. Original examiner imports confirmation and generates final report

3. Sidebar System

Sidebar Container (`app/components/sidebar/sidebar-container.tsx`)

Purpose: Main sidebar wrapper with footer integration

Features:

- Sidebar component orchestration
- Footer modal management
- Keyboard event handling (Escape key)
- Patreon widget integration

Type Definition: Uses `FileData` interface and Firebase `User` type

Key Props:

```
interface SidebarContainerProps {
  user: User;
  onImageSelect: (file: FileData) => void;
  imageId?: string;
  onCaseChange: (caseNumber: string) => void;
  currentCase: string;
  files: FileData[];
  // ... additional props for state management
}
```

Sidebar (`app/components/sidebar/sidebar.tsx`)

Purpose: Core sidebar functionality

Features:

- Case management interface
- File upload and selection
- Image management controls

Type Definition: Uses Firebase `User` type and `FileData` interface

Case Sidebar (`app/components/sidebar/cases/case-sidebar.tsx`)

Purpose: Case-specific sidebar functionality

Features:

- Case creation and management
- File upload interface
- Image selection and deletion
- Case validation and error handling

Type Definition: Uses component-specific `CaseSidebarProps` interface with `FileData` and Firebase `User` types

Key Props:

```
interface CaseSidebarProps {
  user: User;
  onImageSelect: (file: FileData) => void;
  onCaseChange: (caseNumber: string) => void;
  imageLoaded: boolean;
  setImageLoaded: (loaded: boolean) => void;
  onNotesClick: () => void;
  files: FileData[];
  setFiles: React.Dispatch<React.SetStateAction<FileData[]>>;
  caseNumber: string;
  setCaseNumber: (caseNumber: string) => void;
  currentCase: string | null;
  setCurrentCase: (caseNumber: string) => void;
}
```

Notes Sidebar (`app/components/sidebar/notes/notes-sidebar.tsx`)

Purpose: Annotation and notes management interface

Features:

- Comprehensive annotation forms
- Color selection integration
- Classification options (Bullet, Cartridge Case, Other)
- Support level selection (ID, Exclusion, Inconclusive)
- Index type management (number/color)
- Subclass characteristics
- Additional notes handling

Type Definition: Uses component-specific interface with custom types for classification options

Key Props:

```
interface NotesSidebarProps {
  currentCase: string;
  onReturn: () => void;
  user: User;
  imageId: string;
  onAnnotationRefresh?: () => void;
}
```

Data Types:

```
type ClassType = 'Bullet' | 'Cartridge Case' | 'Other';
type IndexType = 'number' | 'color';
type SupportLevel = 'ID' | 'Exclusion' | 'Inconclusive';
```

Cases Modal (`app/components/sidebar/cases/cases-modal.tsx`)

Purpose: Case selection and management modal

Features:

- Paginated case listing

- Case selection interface
- Loading states and error handling
- Keyboard navigation (Escape key)

Type Definition: Uses component-specific `CasesModalProps` interface

Props:

```
interface CasesModalProps {
  isOpen: boolean;
  onClose: () => void;
  onSelectCase: (caseNum: string) => void;
  currentCase: string;
  user: User;
}
```

Files Modal (`app/components/sidebar/files/files-modal.tsx`)

Purpose: File list dialog for the active case with selection and deletion controls.

Features:

- Paginated file listing for large cases
- File selection with active-file highlighting
- Read-only guard for delete operations
- Confirmation status indicators per file
- Keyboard support (Escape to close)

Type Definition: Uses component-specific `FilesModalProps` interface

Props:

```
interface FilesModalProps {
  isOpen: boolean;
  onClose: () => void;
  onFileSelect?: (file: FileData) => void;
  currentCase: string | null;
  files: FileData[];
  setFiles: React.Dispatch<React.SetStateAction<FileData[]>>;
  isReadOnly?: boolean;
  selectedFileId?: string;
}
```

Notes Modal (`app/components/sidebar/notes/notes-modal.tsx`)

Purpose: Additional notes editing modal

Features:

- Text area for detailed notes
- Save/cancel functionality
- Keyboard event handling
- Temporary state management

Type Definition: Uses component-specific `NotesModalProps` interface

Props:

```
interface NotesModalProps {
  isOpen: boolean;
  onClose: () => void;
  notes: string;
  onSave: (notes: string) => void;
}
```

Case Export (`app/components/sidebar/case-export/case-export.tsx`)

Purpose: Comprehensive case data export modal interface with ZIP file support and signature verification utilities

Features:

- Case number input with auto-population from current case
- **Format Selection**: JSON, CSV/Excel, and ZIP export formats with visual toggle
- **ZIP Export with Images**: Single case export with complete data files and associated images
- **Image Inclusion Options**: Checkbox to include/exclude images in ZIP exports
- **Single Case Export**: Export individual case with complete annotation data
- **Bulk Export**: Export all cases with real-time progress tracking
- **Excel Multi-Worksheet**: CSV format creates Excel files with summary and individual case worksheets
- **Comprehensive Data**: All annotation fields including case identifiers, colors, classifications, and split box annotations
- Loading states and error handling with detailed error messages
- Keyboard navigation (Escape key) and accessible controls
- Automatic case number pre-filling when case is loaded
- Progress visualization for bulk export operations with case-by-case updates
- **Synchronized UI States**: Export button and checkboxes disabled appropriately during operations
- **Verification UX**: Dedicated action opens standalone `PublicSigningKeyModal` to view and copy the public signing key
- **Hash Utility Removed**: Verification flow no longer depends on a separate Hash Utility component

Type Definition: Uses component-specific `CaseExportProps` interface

Props:

```
interface CaseExportProps {
  isOpen: boolean;
  onClose: () => void;
  onExport: (caseNumber: string, format: ExportFormat, includeImages?: boo
  onExportAll: (onProgress: (current: number, total: number, caseName: str
  currentCaseNumber?: string;
  isReadOnly?: boolean;
}

export type ExportFormat = 'json' | 'csv';
```

Enhanced Export Features:

- **ZIP Package Creation**: Complete case export with data files and images in structured ZIP archive
- **JSZip Integration**: Browser-based ZIP file generation with progress tracking
- **Image Download System**: Automatic image fetching and packaging for ZIP exports
- **Data Parity**: CSV/Excel exports contain identical data to JSON exports (22+ columns total)
- **Split Box Annotations**: Box annotations split into separate rows for improved data analysis
- **Format Indicators**: Clear UI showing format types with tooltips explaining functionality
- **Progress Callbacks**: Real-time progress updates during bulk export operations
- **Error Recovery**: Graceful handling of failed exports with detailed error reporting
- **Performance Optimization**: Optional annotation inclusion for faster exports when only metadata is needed
- **Disabled State Management**: UI components properly synchronized during export operations
- **Config-Driven Public Key Discovery**: Resolves key ID and PEM from `manifest_signing_public_keys` / fallback config fields
- **Standalone Verification Modal**: Uses `app/components/public-signing-key-modal/public-signing-key-modal.tsx` for key display and copy behavior

Case Import (`app/components/sidebar/case-import/`)

Purpose: Modular ZIP package import system for reviewing complete case data in read-only mode

Architecture: Component composition pattern with custom hooks for business logic separation

Directory Structure:

```
case-import/
├── case-import.tsx          # Main orchestrator component
├── components/              # UI sub-components
│   ├── FileSelector.tsx     # File selection interface
│   ├── CasePreviewSection.tsx   # Case metadata preview
│   ├── ConfirmationPreviewSection.tsx  # Import confirmation details
│   ├── ProgressSection.tsx      # Real-time progress display
│   ├── ExistingCaseSection.tsx  # Existing case management
│   └── ConfirmationDialog.tsx   # Final confirmation modal
├── hooks/                   # Custom business logic hooks
│   ├── useImportState.ts    # State management hook
│   ├── useFilePreview.ts    # File processing hook
│   └── useImportExecution.ts # Import execution hook
├── utils/                   # Pure utility functions
│   └── file-validation.ts   # File type validation
└── index.ts                # Barrel export
```

Component Architecture Features:

- **Single Responsibility**: Each component handles one specific aspect of the import process
- **Custom Hooks Pattern**: Business logic encapsulated in reusable hooks
- **Component Composition**: Main component orchestrates sub-components without complex logic
- **Barrel Exports**: Clean import structure through centralized index.ts
- **Type Safety**: Comprehensive TypeScript interfaces for all component interactions

Main Component Props:

```typescript
interface CaseImportProps {
  isOpen: boolean;
  onClose: () => void;
  onImportComplete?: (result: ImportResult | ConfirmationImportResult) =>
}
```

Custom Hooks:

- **useImportState**: Manages import progress, file selection, and UI state
- **useFilePreview**: Handles ZIP parsing, validation, and preview generation
- **useImportExecution**: Orchestrates the complete import process with progress callbacks

Import Process Features:

- **ZIP File Selection**: Modular file browser interface with validation
- **Read-Only Case Review**: Imported cases automatically protected from modification
- **Progress Tracking**: Real-time import progress with stage-by-stage updates
- **Existing Case Detection**: Automatic detection and management of existing read-only cases
- **Image Integration**: Automatic import and association of all case image data and annotations
- **Metadata Preservation**: Complete preservation of original case metadata and timestamps
- **Clear Management**: Option to remove imported cases from review bin
- **Error Handling**: Comprehensive error reporting with detailed failure messages

- **Security Validation**: Prevents import of cases where user was original analyst
- **ZIP Validation**: Comprehensive validation of ZIP package structure and contents
- **Case Data Parsing**: Support for JSON format with forensic protection warnings
- **Duplicate Prevention**: Prevents import if user was original case analyst
- **Progress Callbacks**: Multi-stage progress reporting (ZIP parsing, image upload, annotation import)
- **Cleanup Operations**: Automatic cleanup of existing case data when overwriting
- **File Mapping**: Internal mapping system for connecting imported images to annotation data
- **Integrity Validation**: Optional case data integrity verification during import
- **User Profile Integration**: Automatic addition of imported cases to user's read-only case list

4. Action Components

Case Management (`app/components/actions/case-manage.ts`)

Purpose: Complete case lifecycle management

Key Functions:

```
export const validateCaseNumber = (caseNumber: string): boolean
export const checkExistingCase = async (
  caseNumber: string,
  user: User
): Promise<boolean>
export const createNewCase = async (
  caseNumber: string,
  user: User
): Promise<{ success: boolean; message: string }>
export const renameCase = async (
  oldCaseNumber: string,
  newCaseNumber: string,
  user: User
): Promise<{ success: boolean; message: string }>
export const deleteCase = async (
  caseNumber: string,
  user: User
): Promise<{ success: boolean; message: string }>
export const listCases = async (user: User): Promise<string[]>
```

Features:

- Case number validation
- Duplicate case detection
- Case creation and deletion
- Case renaming functionality
- User case list management

Case Export (`app/components/actions/case-export/`)

Purpose: Modular case data export system with multi-format support including ZIP packages

Architecture: Organized into specialized modules for maintainability and testability

Directory Structure:

```
case-export/
├── core-export.ts          # Main export orchestration functions
├── data-processing.ts      # Data transformation and CSV generation
├── download-handlers.ts    # Browser download utilities
├── metadata-helpers.ts     # Forensic protection and metadata functions
├── types-constants.ts      # Type definitions and CSV headers
├── validation-utils.ts     # Export validation functions
└── index.ts               # Barrel export
```

Modular Components:

- **Core Export**: Main export functions (`exportCaseData`, `exportAllCases`)
- **Data Processing**: CSV generation, tabular formatting, metadata rows
- **Download Handlers**: Browser-compatible download functions with proper MIME types
- **Metadata Helpers**: Forensic warnings, password protection, Excel worksheet protection
- **Types & Constants**: Shared type definitions and CSV header configurations
- **Validation Utilities**: Case number validation and export prerequisites

Key Functions:

```
// Core export functions
export const exportCaseData = async (
  user: User,
  caseNumber: string,
  options: ExportOptions
): Promise<CaseExportData>

export const exportAllCases = async (
  user: User,
  options: ExportOptions,
  onProgress?: (current: number, total: number, caseName: string) => void
): Promise<AllCasesExportData>

// Download handlers
export const downloadCaseAsJSON = (exportData: CaseExportData): void
export const downloadCaseAsCSV = (exportData: CaseExportData): void
export const downloadCaseAsZip = async (exportData: CaseExportData, includ
export const downloadAllCasesAsJSON = (exportData: AllCasesExportData): vo
export const downloadAllCasesAsCSV = (exportData: AllCasesExportData): voi

// Validation and metadata
export const validateCaseNumberForExport = (caseNumber: string): boolean
export const getUserExportMetadata = (user: User): ExportMetadata
export const addForensicDataWarning = (content: string): string
```

Export Type System:

```
export type ExportFormat = 'json' | 'csv';

export interface ExportOptions {
  includeAnnotations?: boolean;
  format?: 'json' | 'csv' | 'zip';
  includeMetadata?: boolean;
  includeImages?: boolean;
}
```

Enhanced Features:

- **ZIP Export Functionality**: Complete case packaging with data files and images
- **JSZip Integration**: Browser-based ZIP file creation with automatic image downloading
- **Comprehensive Single Case Export**: Complete file and annotation data collection with metadata
- **Bulk Export with Progress**: Export all user cases with real-time progress callbacks and error handling
- **Multi-Format Support**: JSON for structured data, CSV for single cases, Excel (.xlsx) for bulk exports, ZIP for complete packages
- **Excel Multi-Worksheet**: Bulk CSV exports create Excel files with summary worksheet and individual case worksheets
- **Complete Data Parity**: CSV/Excel formats include all annotation fields matching JSON exports
- **Split Box Annotation Format**: Box annotations split into separate rows for improved data analysis

- **Enhanced Box Annotations**: Includes coordinates, colors, timestamps in structured format (label property removed)
- **Forensic Classification Support**: Full case identifiers, color schemes, support levels, and classification data
- **Performance Options**: Configurable annotation inclusion for faster exports when only metadata needed
- **Error Recovery**: Graceful handling of failed case exports with detailed error reporting and continuation
- **File Download Utilities**: Browser-compatible download functions with proper MIME types and cleanup
- **Export Validation**: Comprehensive case number and data validation before export operations
- **Image Management**: Automatic image URL fetching and packaging for ZIP exports

CSV/Excel Export Columns (22+ total with split format):

1. File ID, Original Filename, Upload Date, Has Annotations
2. **Case Identifiers**: Left Case, Right Case, Left Item, Right Item
3. **Visual Elements**: Case Font Color, Index Type, Index Number, Index Color
4. **Classifications**: Class Type, Custom Class, Class Note, Support Level
5. **Options**: Has Subclass, Include Confirmation
6. **Annotations**: Box Annotations Count, Individual Box Annotation Details (split rows with coordinates, colors, timestamps)
7. **Metadata**: Additional Notes, Last Updated

ZIP Export Structure:

- **Data Files**: JSON and CSV formats included in ZIP package
- **Image Directory**: All case images organized in `images/` folder within ZIP
- **Structured Layout**: Professional organization with clear file naming conventions
- **Progress Tracking**: Real-time download progress for images and ZIP creation

XLSX Library Integration:

- **Multi-Worksheet Excel**: Summary sheet plus individual case sheets for bulk exports
- **Structured Data Layout**: Professional formatting with headers and metadata sections
- **Sheet Naming**: Excel-compatible sheet names with case number identifiers
- **Error Sheets**: Dedicated worksheets for failed case exports with error details
- **Split Annotation Format**: Box annotations displayed in separate rows for better analysis

Case Import (`app/components/actions/case-import/`)

Purpose: Modular ZIP package import system for read-only case review and collaboration

Architecture: Organized into specialized modules for complex import operations

Directory Structure:

```
case-import/
├── orchestrator.ts         # Main import orchestration function
├── validation.ts           # Import validation and security checks
├── zip-processing.ts       # ZIP file parsing and preview generation
├── storage-operations.ts   # R2 storage and case management operations
├── image-operations.ts     # Image upload and processing
├── annotation-import.ts    # Annotation data import and mapping
├── confirmation-import.ts  # Confirmation data import processing
└── index.ts               # Barrel export
```

Modular Components:

- **Orchestrator**: Main import workflow coordination (`importCaseForReview`)
- **Validation**: Security checks, exporter UID validation, hash verification
- **ZIP Processing**: Archive parsing, case data extraction, preview generation
- **Storage Operations**: R2 case storage, read-only case management, user profile

updates
- **Image Operations**: Blob processing and upload to image worker
- **Annotation Import**: Complete annotation data mapping and storage
- **Confirmation Import**: Confirmation data processing with integrity validation

Key Functions:

```
// Main orchestrator
export const importCaseForReview = async (
  user: User,
  zipFile: File,
  options: ImportOptions = {},
  onProgress?: (stage: string, progress: number, details?: string) => void
): Promise<ImportResult>

// Validation functions
export const validateExporterUid = async (exporterUid: string, currentUser
export const validateConfirmationHash = (jsonContent: string, expectedHash
export const validateCaseIntegrity = (caseData: CaseExportData, imageFiles

// ZIP processing
export const previewCaseImport = async (zipFile: File): Promise<CaseImport
export const parseImportZip = async (zipFile: File): Promise<{ caseData: C

// Storage operations
export const checkReadOnlyCaseExists = async (user: User, caseNumber: stri
export const addReadOnlyCaseToUser = async (user: User, metadata: ReadOnly
export const storeCaseDataInR2 = async (user: User, caseNumber: string, ca
export const listReadOnlyCases = async (user: User): Promise<ReadOnlyCaseM
export const deleteReadOnlyCase = async (user: User, caseNumber: string):

// Import operations
export const uploadImageBlob = async (blob: Blob, filename: string, user:
export const importAnnotations = async (user: User, caseNumber: string, fi
export const importConfirmationData = async (user: User, confirmationData:
```

Import Type System:

```typescript
export interface ImportOptions {
  overwriteExisting?: boolean;
  validateIntegrity?: boolean;
  preserveTimestamps?: boolean;
}

export interface ImportResult {
  success: boolean;
  caseNumber: string;
  isReadOnly: boolean;
  filesImported: number;
  annotationsImported: number;
  errors?: string[];
  warnings?: string[];
}

export interface ReadOnlyCaseMetadata {
  caseNumber: string;
  importedAt: string;
  originalExportDate: string;
  originalExportedBy: string;
  sourceHash?: string;
  isReadOnly: true;
}

export interface CaseImportPreview {
  caseNumber: string;
  fileCount: number;
  annotationCount: number;
  exportDate: string;
  exportedBy: string;
  hasImages: boolean;
  canImport: boolean;
  warnings: string[];
}
```

Core Import Features:

- **Complete ZIP Package Import**: Full case data and image import from exported ZIP packages
- **Read-Only Protection**: Imported cases are automatically set to read-only mode for secure review
- **Duplicate Prevention**: Prevents import if user was the original case analyst
- **Progress Tracking**: Multi-stage progress reporting with detailed status updates
- **Image Integration**: Automatic upload and association of all case images
- **Metadata Preservation**: Complete preservation of original export metadata and timestamps
- **Data Integrity**: Comprehensive validation of ZIP contents and case data structure
- **Forensic Warning Handling**: Proper removal of forensic warnings for hash validation
- **Confirmation Data Support**: Specialized import handling for confirmation data files
- **Error Recovery**: Graceful handling of import failures with detailed error reporting
- **Security Validation**: Prevents modification of imported cases and restricts access appropriately

Confirmation Export (`app/components/actions/confirm-export.ts`)

Purpose: Confirmation data storage, retrieval, and export with forensic integrity

Architecture: Specialized module for managing authenticated confirmation data in the forensic workflow

Key Functions:

```
// Store confirmation
export async function storeConfirmation(
  user: User,
  caseNumber: string,
  currentImageId: string,
  confirmationData: ConfirmationData,
  originalImageFileName?: string
): Promise<boolean>

// Retrieve confirmations
export async function getCaseConfirmations(
  user: User,
  caseNumber: string
): Promise<CaseConfirmations | null>

export async function getImageConfirmations(
  user: User,
  caseNumber: string,
  originalImageId: string
): Promise<ConfirmationData[]>

export async function getCaseDataWithManifest(
  user: User,
  caseNumber: string
): Promise<{ confirmations: CaseConfirmations | null; forensicManifestCrea

// Export confirmation data
export async function exportConfirmationData(
  user: User,
  caseNumber: string
): Promise<void>
```

Core Features:

- **Confirmation Storage**: Stores authenticated confirmation data linked to original image IDs
- **Multi-Confirmation Support**: Multiple confirmations per image (multiple reviewing examiners)
- **Original Image ID Mapping**: Maps current image IDs to original export image IDs for traceability
- **Forensic Data Export**: Exports confirmations as JSON with SHA256 hash for integrity verification
- **Metadata Integration**: Includes user credentials, timestamps, and forensic manifest linking
- **Case Data Integration**: Confirmations stored within case data structure for unified management
- **Audit Trail Logging**: Comprehensive logging of all confirmation operations (creation, export, failures)
- **Workflow Tracking**: Integrated with audit service workflow system for forensic accountability

Data Structures:

```typescript
interface CaseConfirmations {
  [originalImageId: string]: ConfirmationData[];
}

interface CaseDataWithConfirmations extends CaseData {
  confirmations?: CaseConfirmations;
  forensicManifestCreatedAt?: string;
}

interface ConfirmationExportData {
  metadata: {
    caseNumber: string;
    exportDate: string;
    exportedBy: string;
    exportedByUid: string;
    exportedByName: string;
    exportedByCompany: string;
    totalConfirmations: number;
    version: string;
    originalExportCreatedAt?: string;
    hash: string;
  };
  confirmations: CaseConfirmations;
}
```

Export File Format:

- **Filename Pattern**: `confirmation-data-{caseNumber}-{timestamp}.json`
- **Timestamp Format**: `YYYYMMDD-HHMMSS` in local timezone
- **Content Type**: `application/json` with pretty-printed formatting
- **Hash Algorithm**: SHA-256 secure hash calculated from JSON payload
- **Forensic Linking**: Includes original export timestamp for traceability

Workflow Integration:

1. **Confirmation Creation**:
   - Reviewing examiner confirms findings via Confirmation modal
   - `storeConfirmation` called with confirmation data
   - Original image ID mapping established
   - Confirmation added to case data array
   - Audit event logged with image metadata
2. **Confirmation Retrieval**:
   - Original examiner retrieves confirmations via `getCaseConfirmations`
   - Image-specific confirmations via `getImageConfirmations`
   - Forensic manifest metadata via `getCaseDataWithManifest`
3. **Confirmation Export**:
   - Export triggered from reviewing examiner's system
   - Complete confirmation data collected with metadata
   - SHA-256 hash calculated for forensic integrity
   - JSON file downloaded with hash included
   - Audit trail logged with file size and confirmation count
4. **Import Process** (handled by case-import module):
   - Confirmation JSON file imported by original examiner
   - Hash validated against calculated hash
   - Confirmation data merged into original case
   - Forensic linking preserved via original export timestamp

Security Features:

- **User Identity Validation**: Captures confirming examiner's UID and email
- **Tamper Detection**: SHA-256 hash enables integrity verification
- **Immutable Records**: Confirmations cannot be modified once created
- **Audit Trail**: All operations logged for compliance and traceability
- **Original Image Linking**: Prevents confirmation misattribution via ID mapping

Error Handling:

- Validation of case existence before operations
- Original image ID lookup with fallback mechanisms
- Graceful handling of missing confirmation data
- Comprehensive error logging in audit trail
- User metadata retrieval with fallback defaults

Performance Considerations:

- Processing time tracked for all operations
- File size calculated for audit logging
- Minimal network calls via centralized data operations
- Efficient JSON serialization and hash calculation

Image Management (`app/components/actions/image-manage.ts`)

Purpose: Image upload and retrieval operations

Key Functions:

```
export const uploadImage = async (
  file: File,
  caseNumber: string,
  apiKey: string
): Promise<UploadResult>

export const getImageUrl = async (
  imageId: string,
  apiKey: string
): Promise<string>

export const deleteImage = async (
  imageId: string,
  apiKey: string
): Promise<void>

export const fetchFiles = async (
  caseNumber: string,
  apiKey: string
): Promise<FileData[]>

export const uploadFile = async (
  file: File,
  caseNumber: string,
  apiKey: string
): Promise<{ success: boolean; message: string; fileData?: FileData }>

export const deleteFile = async (
  fileId: string,
  caseNumber: string,
  apiKey: string
): Promise<{ success: boolean; message: string }>
```

PDF Generation (`app/components/actions/generate-pdf.ts`)

Purpose: PDF report generation

Features:

- Dynamic PDF creation
- Annotation integration
- Custom formatting
- Error handling and progress feedback

Key Function:

```
export const generatePDF = async (
  imageUrl: string,
  caseNumber: string,
  annotationData: AnnotationData | null,
  activeAnnotations: Set<string>,
  firstName?: string
): Promise<{ success: boolean; message: string }>
```

Notes Management (`app/components/actions/notes-manage.ts`)

Purpose: Annotation and notes data management

Features:

- CRUD operations for annotation data
- Data validation and sanitization
- Error handling for API operations

Key Functions:

```
export const getNotes = async (
  caseNumber: string,
  imageId: string,
  apiKey: string
): Promise<AnnotationData | null>

export const saveNotes = async (
  caseNumber: string,
  imageId: string,
  notesData: AnnotationData,
  apiKey: string
): Promise<{ success: boolean; message: string }>
```

Sign Out (`app/components/actions/signout.tsx`)

Purpose: User authentication logout

Features:

- Firebase sign out
- Local storage cleanup
- Redirect handling
- Error handling

Type Definition: Uses component-specific `SignOutProps` interface

Props:

```
interface SignOutProps {
  redirectTo?: string;
}
```

5. UI Components

Button System (`app/components/button/`)

Purpose: Reusable button components

Components:

- `Button` - Standard button with variants

Type Definition: Uses component-specific `ButtonProps` interface

Props:

```typescript
interface ButtonProps {
  children: React.ReactNode;
  onClick?: () => void;
  disabled?: boolean;
  type?: 'button' | 'submit' | 'reset';
  variant?: 'primary' | 'secondary' | 'danger';
  size?: 'small' | 'medium' | 'large';
  className?: string;
}
```

Color System (`app/components/colors/colors.tsx`)

Purpose: Color selection interface

Features:

- Predefined color palette
- Custom color wheel
- Color validation
- Real-time preview

Type Definition: Uses component-specific `ColorSelectorProps` interface

Props:

```typescript
interface ColorSelectorProps {
  selectedColor: string;
  onColorSelect: (color: string) => void;
}
```

Icon System (`app/components/icon/icon.tsx`)

Purpose: Centralized icon management

Features:

- SVG icon system
- Consistent sizing and styling
- Type-safe icon names
- Available icons: eye, eye-off, class, ID, index, notes, number, print, other unused/misc icons

Type Definition: Uses custom icon type definitions for type-safe icon selection

Usage:

```tsx
<Icon icon="eye" />
<Icon icon="eye-off" />
```

Public Signing Key Modal (`app/components/public-signing-key-modal/public-signing-key-modal.tsx`)

Purpose: Dedicated dialog for displaying and copying the Striae public signing key used to verify export signatures.

Features:

- Modal-based key display isolated from export form logic
- Displays configured key ID when available
- Read-only PEM textarea with graceful empty-state messaging
- Clipboard copy with fallback for older browser APIs
- Keyboard and overlay dismissal handling (`Escape`, outside click)
- Verification guidance aligned with signature-based workflows (no Hash Utility dependency)

Type Definition: Uses component-specific `PublicSigningKeyModalProps` interface

Props:

```
interface PublicSigningKeyModalProps {
  isOpen: boolean;
  onClose: () => void;
  publicSigningKeyId?: string | null;
  publicKeyPem?: string | null;
}
```

Integration:

- Invoked from `app/components/sidebar/case-export/case-export.tsx` via the "View Public Signing Key" action.
- Key values are resolved from app config and signature utilities before rendering the modal.

Toast System (`app/components/toast/toast.tsx`)

Purpose: User feedback and notifications

Features:

- Success and error message display
- Auto-dismiss functionality
- Customizable styling

Type Definition: Uses component-specific `ToastProps` interface

Props:

```
interface ToastProps {
  message: string;
  type: 'success' | 'error' | 'warning';
  isVisible: boolean;
  onClose: () => void;
}
```

Toolbar (`app/components/toolbar/toolbar.tsx`)

Purpose: Main application toolbar

Features:

- Tool selection management (number, class, index, id, notes, box, print, visibility)
- PDF generation controls
- Visibility toggle
- Active tool state tracking
- Box annotation mode with color selector integration

Type Definition: Uses component-specific `ToolbarProps` interface and `ToolId` type

Props:

```
interface ToolbarProps {
  onToolSelect?: (toolId: ToolId, active: boolean) => void;
  onGeneratePDF?: () => void;
  canGeneratePDF?: boolean;
  onVisibilityChange?: (visible: boolean) => void;
  isGeneratingPDF?: boolean;
}

type ToolId = 'number' | 'class' | 'index' | 'id' | 'notes' | 'print' | 'v
```

Theme Provider (`app/components/theme-provider/theme-provider.tsx`)

Purpose: Application theme management

Features:

- Theme context provision
- Theme persistence
- System theme detection
- Theme switching functionality

Type Definition: Uses custom `Theme` type definition from theme.ts

Theme Types (`app/components/theme-provider/theme.ts`):

```
type Theme = 'light' | 'dark' | 'system';
```

6. User Management Components

User Profile Management (`app/components/user/manage-profile.tsx`)

Purpose: Comprehensive user profile management

Features:

- Profile information editing (display name)
- Email address viewing (read-only)
- Company information viewing (read-only)
- Password change functionality
- User reauthentication
- Firebase integration
- Error handling with detailed messages

Type Definition: Uses component-specific `ManageProfileProps` interface

Props:

```
interface ManageProfileProps {
  isOpen: boolean;
  onClose: () => void;
}
```

Key Features:

- Display name modification
- Company information management (read-only)
- Email address display (read-only)
- User permission status loading
- Account deletion functionality
- Firebase error handling integration

Delete Account (`app/components/user/delete-account.tsx`)

Purpose: Secure account deletion with permission-based restrictions

Features:

- User account deletion with confirmation
- Demo account protection (deletion disabled for `permitted=false`)
- Dual confirmation requirements (UID + email)
- Conditional messaging based on account type
- Email notifications on successful deletion
- Firebase authentication integration
- Automatic logout after deletion

Type Definition: Uses component-specific `DeleteAccountProps` interface with user object type

Props:

```typescript
interface DeleteAccountProps {
  isOpen: boolean;
  onClose: () => void;
  user: {
    uid: string;
    displayName: string | null;
    email: string | null;
  };
  company: string;
  permitted: boolean;
}
```

Permission-Based Behavior:

- **Regular Accounts** (`permitted=true`): Full deletion functionality with standard warnings
- **Demo Accounts** (`permitted=false`): Deletion disabled with informational messaging

Security Features:

- Requires exact UID confirmation
- Requires exact email address confirmation
- Demo account protection
- API key authentication for deletion requests

Inactivity Warning (`app/components/user/inactivity-warning.tsx`)

Purpose: Session timeout management

Features:

- Inactivity detection
- Warning countdown display
- Session extension handling

Type Definition: Uses custom hook types from `useInactivityTimeout` hook

Component State Management

Local State Patterns

Most components use React's built-in state management:

```typescript
// Typical state structure
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState<string | undefined>();
const [data, setData] = useState<DataType | null>(null);
```

Context Usage

AuthContext (`app/contexts/auth.context.ts`)

Purpose: Global authentication state

Provided Values:

- Current user information
- Authentication status
- Login/logout functions

Custom Hooks

useInactivityTimeout (`app/hooks/useInactivityTimeout.ts`)

Purpose: Session inactivity management

Features:

- Configurable timeout periods
- Activity detection
- Automatic logout on timeout

Business Logic Hooks Pattern

The application implements a custom hooks pattern for encapsulating complex business logic, as demonstrated in the case-import system:

useImportState (`app/components/sidebar/case-import/hooks/useImportState.ts`)

Purpose: Centralized state management for import workflow

Pattern: Encapsulates all stateful logic for the import process

Returns:

```
interface ImportState {
  // File management
  selectedFile: File | null;
  setSelectedFile: (file: File | null) => void;

  // Progress tracking
  isImporting: boolean;
  importProgress: number;
  progressStage: string;

  // Case management
  existingCase: string | null;
  previewData: CaseData | null;

  // UI state
  showConfirmation: boolean;
  setShowConfirmation: (show: boolean) => void;
}
```

Benefits:

- Consolidates related state in one hook
- Provides clean API for components
- Enables easy testing of state logic

useFilePreview (`app/components/sidebar/case-import/hooks/useFilePreview.ts`)

Purpose: File processing and validation logic

Pattern: Handles asynchronous file operations with error handling

Functionality:

```
interface FilePreviewHook {
  processFile: (file: File) => Promise<ProcessResult>;
  validateZipStructure: (zipData: JSZip) => ValidationResult;
  extractCaseData: (zipData: JSZip) => Promise<CaseData>;
  previewImages: (zipData: JSZip) => Promise<ImagePreview[]>;
}
```

Benefits:

- Isolates complex file processing logic
- Provides reusable validation functions
- Handles error states consistently

useImportExecution (`app/components/sidebar/case-import/hooks/useImportExecution.ts`)

Purpose: Orchestrates the complete import process

Pattern: Manages side effects and external API calls

Process Management:

```
interface ImportExecutionHook {
  executeImport: (importData: ImportData) => Promise<ImportResult>;
  uploadImages: (images: ImageData[]) => Promise<UploadResult[]>;
  saveCaseData: (caseData: CaseData) => Promise<SaveResult>;
  updateUserProfile: (caseNumber: string) => Promise<UpdateResult>;
  reportProgress: (stage: string, progress: number) => void;
}
```

Benefits:

- Separates API calls from UI components
- Provides consistent progress reporting
- Enables comprehensive error handling

Custom Hooks Best Practices

Single Responsibility: Each hook handles one aspect of business logic

```
// ✅ Good: Focused responsibility
const useFileValidation = (file: File) => {
  // Only handles file validation logic
};

// ❌ Avoid: Mixed concerns
const useFileAndUserManagement = (file: File, user: User) => {
  // Handles both file operations AND user management
};
```

Return Object Pattern: Return objects for multiple values

```
// ✅ Good: Named returns
const useImportState = () => {
  return {
    isLoading,
    error,
    data,
    refetch
  };
};

// ❌ Avoid: Array returns for complex state
const useImportState = () => [isLoading, error, data, refetch];
```

Error Handling: Consistent error handling patterns

```typescript
const useAsyncOperation = () => {
  const [error, setError] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);

  const execute = async (params: OperationParams) => {
    setIsLoading(true);
    setError(null);

    try {
      const result = await performOperation(params);
      return result;
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Unknown error');
      throw err;
    } finally {
      setIsLoading(false);
    }
  };

  return { execute, isLoading, error };
};
```

Component Communication Patterns

Props Down, Events Up

Components follow React's unidirectional data flow:

```typescript
// Parent component
const [selectedImage, setSelectedImage] = useState<string>();

// Child component receives data and callbacks
<ImageSelector
  images={images}
  onImageSelect={setSelectedImage}
/>
```

Event Handling

Components use callback props for communication:

```typescript
interface ComponentProps {
  onSuccess: () => void;
  onError: (message: string) => void;
  onDataChange: (data: DataType) => void;
}
```

Modal and Dialog Patterns

Many components follow consistent modal patterns:

```
// Common modal interface
interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
}

// Keyboard event handling for modals
useEffect(() => {
  const handleEscape = (e: KeyboardEvent) => {
    if (e.key === 'Escape') {
      onClose();
    }
  };

  if (isOpen) {
    document.addEventListener('keydown', handleEscape);
    return () => document.removeEventListener('keydown', handleEscape);
  }
}, [isOpen, onClose]);
```

Styling Approach

CSS Modules

Components use CSS Modules for scoped styling:

```
// Component file
import styles from './component.module.css';

// Usage
<div className={styles.container}>
  <button className={styles.primaryButton}>
    Click me
  </button>
</div>
```

Style Conventions

- **BEM-like naming**: `styles.componentName__elementName--modifier`
- **CSS Custom Properties**: For theming and consistency
- **Intuitive Design**: Clean, simple, user-friendly interfaces
- **Accessibility**: ARIA labels and semantic HTML

Performance Considerations

Component Lifecycle

Components are designed for efficient mounting and unmounting:

```
useEffect(() => {
  // Setup
  const cleanup = setupComponent();

  // Cleanup
  return cleanup;
}, [dependencies]);
```

7. Audit Trail Components

User Audit Viewer (`app/components/audit/user-audit-viewer.tsx`)

Purpose: Comprehensive audit trail viewing and management for forensic accountability

Features:

- Date-range filtering (preset and custom ranges)
- Action/result/case filtering for investigation workflows
- Export functionality (CSV, JSON, signed summary reports)
- Case-specific audit trail summary generation
- Compliance and security incident visibility
- Loading, empty, and error states with keyboard dismiss support

Type Definition: Uses `ValidationAuditEntry` from audit types

Props:

```
interface UserAuditViewerProps {
  isOpen: boolean;
  onClose: () => void;
  caseNumber?: string;
  title?: string;
}
```

Key Features:

Data Management:

- Fetches audit entries through `auditService` (Audit Worker-backed)
- Implements client-side filtering and sorting
- Builds case-level audit summaries when case filters are applied

Filtering Capabilities:

- Date range filtering (from/to dates)
- Action type filtering (authentication, case operations, data access, etc.)
- Result status filtering (success, failure, warning, blocked)
- Case-specific filtering for workflow tracking

Export Functionality:

- **CSV Export**: Formatted for spreadsheet analysis
- **JSON Export**: Complete structured data with metadata
- **Summary Reports**: Human-readable compliance summaries
- **Filtered Exports**: Respects current filter settings
- **Batch Operations**: Export multiple date ranges

UI Components:

- Filter controls with date pickers and dropdowns
- Export buttons with format selection
- Loading states and error handling
- Responsive design for various screen sizes

CSS Classes (from `user-audit.module.css`):

```
.auditViewer          /* Main container */
.controls             /* Filter and export controls */
.filterSection        /* Date and type filters */
.exportSection        /* Export functionality */
.auditTable          /* Data display table */
.auditRow            /* Individual audit entries */
.statusIcon          /* Result status indicators */
.pagination          /* Navigation controls */
.loadingState        /* Loading indicators */
.errorState          /* Error display */
.noData              /* Empty state display */
```

Usage Example:

```
import { UserAuditViewer } from '~/components/audit/user-audit-viewer';

// Basic usage - current user's audit trail
<UserAuditViewer
  isOpen={showAudit}
  onClose={() => setShowAudit(false)}
/>

// Case-specific audit viewing
<UserAuditViewer
  isOpen={showCaseAudit}
  onClose={() => setShowCaseAudit(false)}
  caseNumber="CASE-2024-001"
/>
```

Audit Export Service (`app/services/audit-export.service.ts`)

Purpose: Handles audit data export functionality with multiple format support

Features:

- Multiple export formats (CSV, JSON, Summary)
- Configurable data filtering and formatting
- Browser-compatible file downloads
- Batch export operations
- Custom date range exports
- Compliance report generation

Export Methods:

```
interface AuditExportService {
  exportToCSV: (entries: ValidationAuditEntry[], filename: string, context
  exportToJSON: (entries: ValidationAuditEntry[], filename: string, contex
  exportAuditTrailToCSV: (auditTrail: AuditTrail, filename: string, contex
  exportAuditTrailToJSON: (auditTrail: AuditTrail, filename: string, conte
  generateReportSummary: (auditTrail: AuditTrail, context: AuditExportCont
  appendSignedReportIntegrity: (reportContent: string, context: AuditExpor
  generateFilename: (type: 'case' | 'user', identifier: string, format: 'c
}
```

CSV Export Format:

- Headers: Timestamp, User, Action, Result, Details, Case Number
- ISO 8601 timestamp formatting
- Escaped special characters for spreadsheet compatibility
- UTF-8 encoding with BOM for Excel compatibility

JSON Export Format:

- Complete audit entry objects with all metadata
- Structured hierarchy for programmatic processing
- Timestamps in ISO 8601 format
- Includes audit summary metadata

Summary Report Format:

- Executive summary with key metrics
- Compliance status assessment
- Security incident reporting
- User activity breakdown
- Time-based analysis

Usage Example:

```
import { auditExportService } from '~/services/audit-export.service';

const context = {
  user,
  scopeType: 'case' as const,
  scopeIdentifier: 'CASE-2024-001',
  caseNumber: 'CASE-2024-001'
};

// Export filtered entries to CSV
const filtered = entries.filter(entry =>
  entry.action === 'case-create' &&
  entry.result === 'success'
);
await auditExportService.exportToCSV(filtered, 'case-creation-audit.csv',

// Generate signed summary report text
const summaryText = await auditExportService.generateReportSummary(auditTr
console.log(summaryText);
```

Integration Pattern:

```
// Component integration
const handleExport = async (format: 'csv' | 'json' | 'summary') => {
  const entries = await fetchAuditEntries(filters);
  const context = {
    user,
    scopeType: 'user' as const,
    scopeIdentifier: user.uid
  };

  switch (format) {
    case 'csv':
      await auditExportService.exportToCSV(entries, 'audit-export.csv', co
      break;
    case 'json':
      await auditExportService.exportToJSON(entries, 'audit-export.json',
      break;
    case 'summary':
      // Generate summary report (text format, not file download)
      const summary = await auditExportService.generateReportSummary(audit
      console.log(summary); // Display in UI or process as needed
      break;
  }
};
```

Accessibility Features

Built-in Accessibility

- **Semantic HTML**: Proper element usage
- **ARIA Labels**: Screen reader support
- **Keyboard Navigation**: Full keyboard accessibility
- **Focus Management**: Proper focus handling
- **Color Contrast**: WCAG compliance

Development Guidelines

Component Creation Checklist

1. □ Create component directory
2. □ Implement TypeScript interfaces

3. □ Add CSS Module styling
4. □ Include error handling (follow Error Handling Guide)
5. □ Add loading states
6. □ Implement accessibility features
7. □ Add documentation

Best Practices

- **Single Responsibility**: Each component has one clear purpose
- **Type Safety**: Full TypeScript coverage
- **Error Boundaries**: Graceful error handling (see Error Handling Guide)
- **Performance**: Optimized for large datasets
- **Maintainability**: Clear, documented code